Bacharelado em CIÊNCIA DA COMPUTAÇÃO

Desenvolvimento Web I
Prof. Hewerton Enes de Oliveira
2026/2
Lista de Exercícios – CSS: Flexbox (Parte 1)

Responda com atenção. Nas questões dissertativas e práticas, justifique suas respostas.

Parte A – Múltipla escolha

1. Qual valor de `display` ativa o modelo Flexbox em um elemento, tornando-o um **flex container**?

   (a) `display: block`

   (b) `display: inline`

   (c) `display: flex`

   (d) `display: grid`

2. Em um flex container com `flex-direction: row`, o **eixo principal** é orientado:

   (a) de cima para baixo

   (b) da esquerda para a direita (por padrão)

   (c) sempre da direita para a esquerda

   (d) depende da largura do navegador

3. Qual é o valor padrão da propriedade `flex-direction`?

   (a) `column`

   (b) `row-reverse`

   (c) `row`

   (d) `column-reverse`

4. Qual valor de `flex-wrap` faz os itens **quebrarem para uma nova linha** quando não cabem no container?

   (a) `nowrap`

   (b) `wrap`

   (c) `wrap-reverse`

   (d) `break`

5. A propriedade `flex-flow` é um atalho para:

   (a) `flex-direction` + `flex-wrap`

(b) `flex-grow` + `flex-shrink` + `flex-basis`

(c) `justify-content` + `align-items`

(d) `gap` + `margin`

6. A propriedade `justify-content` alinha os itens:

   (a) no eixo cruzado

   (b) no eixo principal

   (c) sempre verticalmente

   (d) apenas quando há um único item

7. Qual valor de `justify-content` distribui os itens com o **primeiro no início**, o **último no fim** e **espaços iguais** entre eles?

   (a) `space-around`

   (b) `space-evenly`

   (c) `space-between`

   (d) `center`

8. Para centralizar um único item **horizontal e verticalmente** dentro de um container flex, qual combinação é suficiente?

   (a) `justify-content: center;` e `align-items: center;`

   (b) apenas `margin: auto;`

   (c) apenas `align-self: center;`

   (d) `display: block;` e `margin: 0 auto;`

9. Qual é o valor padrão da propriedade `align-items`?

   (a) `flex-start`

   (b) `center`

   (c) `stretch`

   (d) `baseline`

10. Qual propriedade permite **sobrescrever** o alinhamento do eixo cruzado para **um único item**, sem afetar os demais?

    (a) `align-items`

    (b) `align-content`

    (c) `align-self`

    (d) `justify-content`

Parte B – Dissertativas e análise de código

11. Explique, com suas palavras, a diferença entre o **eixo principal** (*main axis*) e o **eixo cruzado** (*cross axis*) em um flex container.

12. Considere o código abaixo. Descreva **como** os itens ficarão dispostos visualmente e **onde** cada um estará posicionado.

```html
<div class="painel">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>
```

```css
.painel {
  display: flex;
  flex-direction: row;
  justify-content: space-around;
  align-items: center;
  height: 200px;
  border: 1px solid \#999;
}
.painel > div {
  width: 80px;
  height: 60px;
  background: \#00539c;
  color: \#fff;
}
```

13. Qual a diferença entre `flex-direction: row` e `flex-direction: column`? O que muda no papel de `justify-content` e `align-items`?

14. Qual a diferença entre `align-items` e `align-content`? Em qual situação o `align-content` passa a ter efeito visível?

15. Explique o papel de `flex-grow`, `flex-shrink` e `flex-basis`, e escreva o que o atalho `flex: 2 1 200px` significa.

Parte C – Prática

16. Escreva o CSS (e o HTML mínimo) necessário para centralizar uma `div` de 200×100px **horizontal e verticalmente** na viewport, usando Flexbox.

17. Crie uma **barra de navegação horizontal** com 4 itens: Início, Sobre, Serviços e Contato. Os itens devem ficar **igualmente distribuídos** (espaços iguais nas pontas e entre eles), com fundo azul IFC (`#00539c`) e texto branco. Use Flexbox.

18. Construa um layout com **3 cards em linha**, cada um com 240px de largura, espaçamento de 16px entre eles. Se a tela for pequena (não couberem todos na mesma linha), os cards devem **quebrar para a próxima linha** em vez de serem espremidos.

19. Crie uma **galeria responsiva** com 6 itens que: (a) ficam lado a lado preenchendo o espaço disponível, (b) crescem igualmente para ocupar a linha toda e (c) quebram para novas linhas quando a tela for estreita. Use `flex-wrap` e `flex-grow`.

20. *(Desafio)* Construa um **cabeçalho** com logo à esquerda e menu à direita. Em seguida, usando `order` e `align-self`, faça: (a) o item "Contato" aparecer **antes** dos demais no menu, apenas visualmente, e (b) o logo ficar **alinhado ao topo** enquanto o menu fica **centralizado verticalmente**.